EECS3311 Stage 2 Implementation Report: Real Estate Advisory Agent

Mohammad Islam · Student ID 219284330 · Repository: github.com/idrak888/eecs3311-real-estate-agent

1. Project overview and implementation status

The Real Estate Advisory Agent is a Java 17 desktop application that helps a buyer search a local listing dataset, rank and compare properties, check mortgage affordability, keep a shortlist, and talk to an AI advisor in plain English. It implements the Stage 1 design: a layered architecture (UI > Service facade > Domain / Agent > Persistence), with the LLM used only where language understanding is needed and every number produced by deterministic code.

Repository: https://github.com/idrak888/eecs3311-real-estate-agent

Build: Maven (Jackson 2.17.2 for JSON, JUnit 5.10.3). Entry point `ca.yorku.realestate.app.Main`; GUI by default, CLI with `--cli`

Status at submission: all ten Stage 1 features (F01–F10) are implemented and reachable from both the Swing GUI and the CLI. The suite has 326 JUnit tests across domain, persistence, service, agent and UI-model classes, and the agent has been run against the live Anthropic API.

Area	Status
Data import (JSON + CSV, validation, rejection summary)	Complete
Search, filters, ranking, criteria editing	Complete
Mortgage and affordability	Complete
Comparison, shortlist with undo/redo, persistence	Complete
LLM agent (extraction, tool selection, memory, grounding check)	Complete
Swing GUI and CLI	Complete
Stage 2 UML (diagrams/stage2/)	Complete
Full test plan and KUMA agent validation	Stage 3

The dataset is 60 synthetic Greater Toronto Area listings in `src/main/resources/data/` (JSON and CSV). Three listings (L014, L038, L053) deliberately omit square footage to exercise the "unavailable" paths.

2. Feature implementation mapping

Each feature keeps its Stage 1 ID and classification. Video links are filled in Section 7.

ID	Feature	Main classes / methods	Patterns	Type
F01	Listing data import	`ListingImporter.importFile`, `ListingParserFactory.createParser`, `JsonListingParser`, `CsvListingParser`, `ListingRepository`, `LoadSummary`	Factory Method	Deterministic
F02	Natural-language search	`AdvisorAgent.handleQuery`, `selectTool`, `RequirementExtractor.extractRequest` → `ExtractedRequest`, `MemoryManager.mergeCriteria` / `mergePreferences`, `SearchTool`, `ZeroMatchAnalyzer`	Adapter, Facade	Hybrid
F03	Structured criteria filtering	`SearchCriteriaBuilder`, `SearchCriteria.toFilter`, `AndFilter` / `OrFilter` + six leaf filters, `ListingRepository.findMatching`, `CriteriaPanel`	Builder, Composite	Deterministic
F04	Result ranking	`RankingStrategy.rank`; `PriceAscendingStrategy`, `PriceDescendingStrategy`, `ValuePerSqFtStrategy`, `BestMatchStrategy`	Strategy	Deterministic
F05	Property detail view	`RealEstateService.getListing`, `ListingDetailDialog`, CLI `show`, `Listing.getPricePerSqFt`	Facade	Deterministic
F06	Mortgage calculation	`MortgageCalculator.calculate`, `MortgageTerms`, `MortgageResult`, `MortgageTool`	–	Deterministic
F07	Affordability analysis	`MortgageCalculator.assess`, `maxSupportablePrice`, `FinancialProfile`,	–	Deterministic

ID	Feature	Main classes / methods	Patterns	Type
		AffordabilityResult, Verdict, ProfileDialog		
F08	Property comparison	ComparisonTool, ComparisonResult, AdvisorAgent.compareProperties, RealEstateService.compare, ComparisonDialog	Facade	Hybrid
F09	Shortlist management	Shortlist, AddToShortlist, RemoveFromShortlist, CommandInvoker (undo/redo), PreferenceStore, ShortlistPanel	Command	Deterministic
F10	Trade-off explanation	AdvisorAgent.explainResults, validateGrounding, one regeneration, LLMClient	Adapter	AI

How the AI is bounded. In F02, F08 and F10 the model never computes anything. It turns text into criteria (validated against a fixed vocabulary), picks one of three tools (validated, falling back to search), and writes explanations about listings the tools already returned. Every listing ID in a reply is checked against the supplied set; one failure triggers a regeneration, a second drops the narrative and shows the deterministic results with a notice.

3. Design patterns and updated UML

All eight Stage 1 patterns were implemented as designed. The table shows where each lives and what it does in the running code.

Pattern	Classes	What it does here
Facade	RealEstateService	GUI and CLI call one object; neither touches repositories, filters or the agent directly, so both UIs share every behaviour.
Strategy	RankingStrategy + 4 strategies	Ranking mode switches at runtime (combo box or --sort) without changing search code.
Composite	Filter, CompositeFilter, AndFilter, OrFilter; leaves PriceFilter, BedroomFilter, BathroomFilter, TypeFilter,	SearchCriteria.toFilter builds a tree (OR across chosen types or areas, AND

Pattern	Classes	What it does here
	NeighborhoodFilter, FeatureFilter	across fields); ZeroMatchAnalyzer rebuilds it with one requirement removed.
Command	ShortlistCommand, AddToShortlist, RemoveFromShortlist, CommandInvoker	Every shortlist change can be undone and redone, and is persisted after execution.
Factory Method	ListingParserFactory, JsonListingParser, CsvListingParser	Format is chosen by file extension; a new format is one register call (tested with a TSV stub).
Observer	SearchObserver; observers MainWindow, CLIRenderer	Results, criteria, shortlist and warning changes push to whichever UI is running.
Adapter	LLMClient, AnthropicLLMClient, MockLLMClient	Agent code never sees HTTP; tests use the mock, the app uses the real API.
Builder	SearchCriteriaBuilder	Criteria from the panel, the extractor and the CLI are built and validated the same way.

Updated diagrams. The Stage 2 UMLet files are in diagrams/stage2/, with a README listing every change per diagram. Stage 2 additions are shaded green.

- Diagram 1a: architecture spine (facade, observer)
- Diagram 1b: search (criteria, builder, composite filters, ranking, ZeroMatchAnalyzer)
- Diagram 1c: commands and financial (shortlist commands, PreferenceStore, mortgage classes)
- Diagram 2a: agent core (AdvisorAgent, MemoryManager, RequirementExtractor, ExtractedRequest, LLM adapter)
- Diagram 2b: agent tools (ToolManager, SearchTool, MortgageTool, ComparisonTool)
- Diagram 3: user interface (new; GUI panels and dialogs, CLIRenderer, Main)
- SD02: natural-language interpretation; SD03: search tool execution

The use-case diagram and SD04 to SD07 did not change.

The diagram sources (UMLet .uxf) are at https://github.com/idrak888/eecs3311-real-estate-agent/tree/main/diagrams/stage2 and open directly in UMLet

4. Changes from the Stage 1 design

No pattern or layer was removed. Changes are additions found necessary while implementing. The full running record is docs/changes.md.

Stage 1 design	Change	Reason	Impact
ListingParser returns listings only	Added getRejections(); new ListingImporter returns a LoadSummary	F01 must report rejected records and why	loadListings returns LoadSummary, not int
Leaf filters for price, bedrooms, type, feature	Added BathroomFilter, NeighborhoodFilter; CompositeFilter base	SearchCriteria had minBathrooms and neighborhoods but nothing evaluated them	Two leaves and one abstract class in Diagram 1b
Builder with four with* methods	Added withMinPrice, withMinBathrooms, withNeighborhood, from, clear*	Every field must be settable from GUI, CLI and LLM output	More builder methods
SearchCriteria.isValid()	Added validationErrors(), describe(), isEmpty()	F03 must say why a search did not run	More methods in Diagram 1b
No defined "most restrictive criterion"	New ZeroMatchAnalyzer drops each requirement and counts matches	F02/F03 must name it; computing it keeps advice true to the data	New domain class used by tool, agent, CLI and GUI
RankingStrategy.score()	Default rank(); BestMatch defined as average per-criterion satisfaction with partial credit	Sort is the same for every strategy; "best match" was undefined	One interface method

Stage 1 design	Change	Reason	Impact
"Standard" affordability thresholds	GDS 39%, TDS 44%, marginal within 3 points; housing = payment + 1%/yr tax + $100 heating	Stage 1 gave no numbers	`AffordabilityResult` gained housing cost, `debtLimited`, explanation
`MortgageCalculator.monthlyPayment`	Added `calculate()` → `MortgageResult`, `monthlyHousingCost()`, `MortgageTerms.defaultsFor()` (20%, 5.0%, 25 yr)	Service needs all F06 outputs at once; F05 shows payment at defaults	Two methods in Diagram 1c
`CommandInvoker.undo()` void	Returns `Optional`; commands report `changedState()`; remove-undo restores position	No-ops must not hit the undo stack; "nothing to undo" must be reported	Changed signatures in Diagram 1c
`PreferenceStore` save/load	Atomic writes; corrupt file throws `IOException`	F09: warn instead of losing data	`SearchObserver.onWarning()` added
Extractor returns `SearchCriteria`	`extractRequest()` returns `ExtractedRequest` (requirements + preferences); vocabulary from loaded data	Live test: "maybe in Vaughan" became a hard filter and returned 0 of 60	`MemoryManager` keeps preferences that rank but do not filter
`selectTool` returns an `AgentTool`	Returns a validated decision (tool, listing IDs, terms) with fallback to search	Model must say which listings "the second one" means	Mortgage text formatted in code, never by the model
`AgentResponse` text + listings	Added mortgage, affordability, notice, `aiAvailable`	UIs must show "AI unavailable" and "explanation unavailable"	GUI status bar and CLI tip
`AnthropicLLMClient` single call	Injectable transport, token counting, one	Testable without network; usage	`EnvConfig`, `LLMException` beside Diagram 2a

Stage 1 design	Change	Reason	Impact
	retry on 429/5xx/529/IO	logging; resilience	
MainWindow holds all controls	Split into CriteriaPanel, ChatPanel, ShortlistPanel + three dialogs; Formats utility; SwingWorker for agent calls	Focused classes; identical numbers in GUI and CLI; responsive window	New Diagram 3

5. AI tools and collaboration log

Claude (claude.ai, model Claude Opus 5.5) was used throughout as a pair programmer: planning commits against F01–F10, generating classes and tests, and reviewing UI screenshots. Every generated change was read, built and tested before it was committed. The running log is docs/ai-log.md.

Development sessions

Date	Tool / model	Task (features / classes)	Tokens (est.)	Cost (USD)
Oct 21	Claude, Opus 5.5	Plan; Maven project, Listing, PropertyType (F05); parsers + factory, repository, importer, dataset (F01)	~400k	$2.88
Oct 22	Claude, Opus 5.5	Tooling: stalled Homebrew Maven install; switched to Eclipse's embedded Maven	~40k	$0.29
Oct 26	Claude, Opus 5.5	Composite filters, findMatching (F03); SearchCriteria, builder (F02/F03)	~250k	$1.80
Oct 30	Claude, Opus 5.5	Ranking strategies (F04); mortgage (F06); affordability (F07)	~350k	$2.52
Oct 30	Claude, Opus 5.5	Shortlist commands (F09), PreferenceStore, facade + observer, LLM adapter, RequirementExtractor	~400k	$2.88

Date	Tool / model	Task (features / classes)	Tokens (est.)	Cost (USD)
Nov 4	Claude, Opus 5.5	`MemoryManager`, tool framework, `ComparisonTool` (F08), `AdvisorAgent` (F02/F10), soft-preference fix	~500k	$3.60
Nov 5	Claude, Opus 5.5	CLI and Swing GUI; layout checked from rendered screenshots	~450k	$3.24
Nov 6	Claude, Opus 5.5	`ZeroMatchAnalyzer`, LLM retry, live AI status	~300k	$2.16
Nov 7	Claude, Opus 5.5	Stage 2 UMLet diagrams and this report	~350k	$2.52
Total			**~3.0M**	$21.89

Token counts are estimates. claude.ai does not show token counts for each session. The figures are approximate, based on the size of the code, files and conversation in each session. Development used a flat-rate Claude subscription, so these sessions were not billed individually.

The application's own LLM use. The agent calls the Anthropic Messages API (model `claude-sonnet-5-5`), billed per token to the developer's API key. `AnthropicLLMClient` counts input and output tokens. The first live check used 686 input and 63 output tokens, under $0.01. Development testing and demo recording together used well under $1 of API credit.

Representative interactions and what was changed by hand

- **Design gap caught before generating code (Oct 26).** Checking Diagram 1b against `SearchCriteria` showed `minBathrooms` and `neighborhoods` had no leaf filter, so they would have been silently ignored. Added `BathroomFilter` and `NeighborhoodFilter`.
- **Wrong test, right code (Oct 30).** A generated test expected a payment of $3,896.58 for $680k at 4.8% over 25 years. Re-deriving it independently in Python gave $3,896.38: the code was right and the test expectation was corrected.
- **Assertion that could never fail (Oct 30).** A generated service test checked `results.containsAll(results)`. Replaced with a check that re-ranking keeps the same listings and orders them by price.

- **Live test changed the design (Oct 31).** "3 bedroom house under 1.5M with a garage, maybe in Vaughan or Woodbridge" extracted correctly but matched 0 of 60, because "maybe" became a hard filter. Hedged wishes now become preferences that rank results without filtering them.
- **Numbers stay in code (Nov 4).** Decided the model never writes mortgage figures. The agent formats payment, interest and verdict from calculator output; the model only picks tools and writes explanations, which are checked for invented listing IDs.
- **GUI layout rejected (Nov 5).** Rendered screenshots showed the criteria panel squeezed ("Townhouse" cut off, lists collapsed to one row) and the comparison table printing "3.0". Both fixed before committing.
- **Advice computed, not generated (Nov 6).** The "most restrictive requirement" for an empty search is computed by re-running the search without each requirement. Expected counts for the Vaughan query (area 3, garage 3, price/beds/type 1 each) were verified in Python before writing the test.

Security. The API key lives only in a local `.env` file, which is git-ignored and has never been committed. `.env.example` holds a placeholder. No key appears in this report, the logs or the videos.

6. Development process and verification

Commits. 25 implementation commits plus one diagrams commit, each building and passing tests on its own. Messages start with the feature IDs they implement.

Period 1 (Oct 20–26): 5 commits

Commit	Date	Message
944782b	Oct 21	F05: Maven project, `Listing` and `PropertyType` with validation
2b15434	Oct 21	F01: JSON/CSV parsers and `ListingParserFactory`
845c7f5	Oct 22	F01: repository, importer with load summary, 60-listing dataset
e8eb419	Oct 26	F03: Composite filter tree and `findMatching`
0fbccbf	Oct 26	F02/F03: `SearchCriteria` and builder

Period 2 (Oct 27–Nov 2): 9 commits

Commit	Date	Message
0d3c29e	Oct 30	F04: ranking strategies
30c1c6f	Oct 30	F06: `MortgageCalculator`

Commit	Date	Message
717f2eb	Oct 30	F07: affordability assessment
9d138aa	Oct 30	F09: shortlist commands and undo
ea97b8e	Oct 30	F07/F09: `PreferenceStore`
6945949	Oct 30	`RealEstateService` facade with observers
6546660	Oct 30	F02: LLM adapter and `.env` config
442e24b	Oct 30	F02: `RequirementExtractor`
af2bbf1	Oct 31	F02: `LlmConnectionCheck` (live API)

Period 3 (Nov 3–9): 11 commits + diagrams

Commit	Date	Message
82804ac	Nov 4	F02/F10: `MemoryManager`
82a465f	Nov 4	F02/F06: tool framework, `SearchTool`, `MortgageTool`
a68b16b	Nov 4	F08: `ComparisonTool`
0cd63b0	Nov 4	F02/F08/F10: `AdvisorAgent`
d10bc6f	Nov 4	F02: agent wired into facade; soft preferences
1610e41	Nov 6	CLI frontend
2a0127f	Nov 6	F01/F03/F04: Swing main window
c8765ab	Nov 6	F02/F10: GUI chat panel
b7761f1	Nov 6	F05–F09: details, shortlist, comparison, undo in GUI
028158b	Nov 6	F02/F03: most restrictive requirement on zero matches
c31a018	Nov 6	F02: LLM retry and live AI status
4394a12	Nov 7	Stage 2 UMLet diagrams

Verification done in Stage 2

- 326 JUnit 5 tests. The agent is tested with `MockLLMClient`, including malformed JSON, unknown tools, invented listing IDs and API failures. The HTTP client is tested through a fake transport.

- Financial expectations were computed independently (Python) rather than copied from the code under test.
- GUI layout checked from rendered screenshots, then tested by hand in Eclipse.
- Live agent run against the Anthropic API (`LlmConnectionCheck`, then GUI and CLI).

Defects found and fixed

Defect	Fix
Bathroom and neighborhood criteria had no filter	Two new leaf filters
Hedged areas ("maybe in Vaughan") filtered everything out	Preferences that rank only
Wrong expected payment in a test	Recomputed independently
`ShortlistPanel.list()` clashed with `Component.list()`	Renamed `listView()`
Criteria panel squeezed; "3.0" in comparison table	Fixed sizing; whole-number formatting
One API overload lost the user's turn	Single retry on 429/5xx/529/IO

Known limitations

- Listings are synthetic; prices and neighborhoods are realistic but not real.
- Mortgage maths uses standard monthly compounding, not Canadian semi-annual compounding, and affordability does not apply a stress-test rate.
- Explanations depend on the external API; when it is down the app falls back to structured search with a notice.
- The full test plan, coverage report and KUMA agent validation are Stage 3 work.

7. Demonstration video index

The features are demonstrated in two videos. Video 1 covers F01–F07 and Video 2 covers F08–F10. Each feature segment starts with its ID on screen,runs the feature in the GUI and then the CLI, and shows at least one error case. No API key is visible in either recording.

- **Video 1 (F01–F07):** https://youtu.be/TFg8pC5BoVI
- **Video 2 (F08–F10):** https://youtu.be/Yu5H5UEdt7Y

ID	Feature	What the segment shows	Error case shown	Video	Starts at
F01	Listing data import	Startup load summary (GUI status bar, CLI banner); `load` of the CSV file	Unsupported file extension	1	0:45
F02	Natural-language search	"3 bedroom house under 1.5M with a garage, maybe in Vaughan or Woodbridge" in chat and `ask`; a follow-up refinement	Zero matches naming the most restrictive requirement	1	1:50
F03	Structured filtering	Criteria panel; `search` flags	Min price above max price	1	4:00
F04	Result ranking	Switching all four strategies; `--sort`	Value ranking with listings missing square footage	1	5:46
F05	Property detail view	Double-click a row; `show L020`	`show` with an unknown ID; L014 price per sq ft "unavailable"	1	6:15
F06	Mortgage calculation	Mortgage tab; `mortgage L020 --down 150k --rate 4.8 --years 25`	Down payment above price	1	6:45
F07	Affordability analysis	Financial profile, then verdicts; `profile` and `afford`	Debt alone over the TDS limit	1	7:55
F08	Property comparison	Compare 3 shortlisted listings with explanation; `compare`	Fewer than 2 or more than 4 selected	2	1:00
F09	Shortlist management	Add, remove, undo, restart and reload; `shortlist`, `undo`	Undo with nothing to undo	2	2:00
F10	Trade-off explanation	Advisor explaining results in chat and `ask`	AI unavailable (no key or network)	2	1:45

ID	Feature	What the segment shows	Error case shown	Video	Starts at
			with structured fallback		

8. Running the application and API key setup

Prerequisites: Java 17 or later, Maven 3.8 or later (or Eclipse with embedded Maven).

API key (local only).

```
cp .env.example .env
# edit .env and replace YOUR_LOCAL_API_KEY_HERE
```

`.env` is git-ignored. Tests never call the real API, so `mvn test` works without a key. Without a key, everything works except chat and `ask`, which report that the assistant is unavailable.

Build and test: `mvn test`

(option 1) Run from Eclipse: open `app/Main.java` and choose Run As → Java Application for the GUI. For the CLI, add `--cli` under Run Configurations → Arguments. Run from the project root so `.env` is found.

(option 2) Run from a terminal:

```
mvn compile exec:java                    # GUI
mvn compile exec:java -Dexec.args="--cli"  # CLI
```

Option `--data path/to/listings.csv` loads a different file.

GUI layout. Left: search criteria and ranking (also filled in by the advisor). Centre: results (double-click for details, mortgage and affordability) and advisor chat. Right: shortlist; select 2–4 and press Compare. Edit → Undo reverses shortlist changes. Profile → Financial profile sets income, debt and down payment.

CLI examples

```
search --max-price 900k --beds 3 --type house --area Vaughan --feature garage --sort value
ask 3 bedroom house under 1.5M, maybe in Vaughan or Woodbridge
show L020
mortgage L020 --down 150k --rate 4.8 --years 25
help
```

User data (shortlist, financial profile) is saved under `data/user/`, which is git-ignored.